Martin Gonzalez Rico

Clases abstractas, interfaces, enums y anidadas

No.	Respuesta	Explicación:
1	d.abstract final class Person { }	abstract y final tienen propósitos contrarios. Una clase abstract está pensada para que otras clases puedan heredar de ella, mientras que una clase final no permite que ninguna clase herede de ella. Por eso Java no permite utilizar ambos modificadores al mismo tiempo. Las demás declaraciones son válidas.
2	a.A class that is abstract cannot be instantiated or turned into an object (instance). c.If a class has abstract methods, then it must be specified as an abstract class.	Una clase abstract no se puede instanciar directamente con new, aunque sí podemos crear objetos de sus subclases concretas. Además, si una clase contiene métodos abstractos, debe declararse también como abstract, porque esos métodos no tienen una implementación completa en esa clase. Una interfaz sí puede extender otra interfaz, pero no puede extender una clase.
3	c.An abstract class can be extended by a concrete class. d.An abstract class can implement an interface	Una clase concreta puede extender una clase abstracta si implementa todos los métodos abstractos que debe implementar. También una clase abstracta puede implementar una interfaz sin implementar inmediatamente todos sus métodos, porque puede dejar esa responsabilidad para una subclase. Recuerda que una interfaz extiende otra interfaz, mientras que una clase implementa una interfaz.
4	a.It compiles without error.	El código compila porque una clase abstracta puede tener atributos, constructores con parámetros y métodos privados. Tampoco está obligada a declarar métodos abstractos; puede tener únicamente métodos normales. Lo importante es que abstract impide crear directamente un objeto de esa clase, pero no significa que todos sus métodos deban ser abstractos.
5	a.The program will print "Drawing a circle" followed by "This is a shape".	Circle implementa el método abstracto draw() de Shape, por lo que ya se puede crear un objeto de Circle. Además, hereda el método normal printShape() de la clase padre. Como primero se llama a circle.draw() y después a circle.printShape(), la salida respeta ese orden.
6	a.Car started Vehicle stopped Bike started Vehicle stopped	La variable car apunta a un objeto Car, así que car.start() ejecuta la versión sobrescrita de Car. Sin embargo, stop() no está sobrescrito en Car, así que se ejecuta el método heredado de Vehicle. Después sucede lo mismo con bike: se ejecuta Bike.start() y luego el stop() de Vehicle.
7	c.MONDAY TUESDAY WEDNESDAY THURSDAY FRIDAY SATURDAY SUNDAY	El método values() devuelve todas las constantes del enum en el orden en que fueron declaradas. El ciclo for-each recorre cada una y System.out.print(day + " ") imprime su nombre

		seguido de un espacio. Por eso aparecen los siete días sin comas y en el orden original.

8	b.Opción b:
	public class Bike implements Vehicle {
	public void start() {
	System.out.println("Bike starts");
	}
	void stop() {
	System.out.println("Bike stops");
	}
	}
		La interfaz Vehicle declara los métodos start() y stop(), que son implícitamente public. En Bike, el método stop() se declara sin modificador de acceso, por lo que tiene acceso por defecto y es menos visible que el método de la interfaz. Java no permite reducir la visibilidad al implementar ese método.

9	d.class Base3 { abstract int var1 = 89; }	abstract se utiliza para declarar clases o métodos abstractos, pero no atributos o variables. Las otras opciones son válidas: puede haber clases internas abstractas y una clase abstracta no necesita obligatoriamente contener métodos abstractos.
10	e.None of the options is valid.	La interfaz I declara void method(), que es un método implícitamente public. Como C no está declarada como abstracta, debe implementar el método con la misma firma, con acceso public y con un cuerpo. Las opciones que usan private o no especifican un modificador reducen la visibilidad; la opción abstract tampoco sirve para completar una implementación normal. Por eso ninguna opción es válida.
11	d.Compilation fails	El código tiene dos errores. El primero es que getGait() implementa un método de la interfaz Rideable, pero no está declarado como public; los métodos de interfaz de este tipo son públicos y no se puede reducir su visibilidad. El segundo error es que el código utiliza Weight++, pero el atributo declarado se llama weight. Por eso no llega a ejecutarse.
12	b.Amarillo Rosa Rojo	Aunque tp está declarado como tipo A2, el objeto real es de tipo A3. Como A3 sobrescribe los métodos m1(), m2() y m3(), Java ejecuta las versiones de esa clase. Por eso se imprime primero Amarillo, después Rosa y finalmente Rojo.
13	d.red green	Cuando el ciclo llega a RED, la condición Ot.Colors.RED.equals(c) se cumple y se imprime red. Cuando llega a GREEN, la condición c == Ot.Colors.GREEN se cumple y se imprime green. La comparación c.equals("BLUE") nunca es verdadera porque c es una constante del tipo Colors y "BLUE" es un String; aunque

		ambos tengan un texto parecido, son tipos diferentes. Por eso no se imprime blue y tampoco ocurre un error.
14	h.Todas las opciones son correctas	Una clase anidada puede utilizar modificadores de acceso como public, protected y private, además de modificadores como final, abstract, static y strictfp. Esto es diferente de una clase de nivel superior, que no puede declararse private, protected ni static. Por eso, en este caso, todas las opciones son válidas para una clase anidada.